

Farmer Authentication & Registration Guide

This guide provides the complete payload documentation for the Farmer Registration and Login APIs, along with a comprehensive architectural blueprint for the Android developer to implement token-based authentication seamlessly.

1. API Reference

1.1 Farmer Registration

Registers a new farmer account and uploads their profile image.

Endpoint: POST /api/user/farmer-registration

Content-Type: multipart/form-data

Request Form Data:

Field	Type	Required	Description
first_name	text	Yes	Farmer's first name
last_name	text	Yes	Farmer's last name
email	text	Yes	Valid email
password	text	Yes	Account password
mobile_number	text	Yes	Exactly 10 digits
dob	text	No	Date of birth (DD/MM/YYYY or YYYY-MM-DD)
isverifyEmail	text	No	"true" or "false"
isMobileVerify	text	No	"true" or "false". <i>At least one verification must be true.</i>
address	text	No	JSON string: [{"state": "1", "district": "2", "block": "3", "lanel": "...", "village": "...", "pincode": "...", "is_primary": true}]
profile_image	file	No	jpg, jpeg, or png (Max 5MB)

Success Response (201 Created)

```
{
  "success": true,
  "message": "Farmer registered successfully",
  "user_id": 123,
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

1.2 Login API

Authenticates a user and provides both an `accessToken` (short-lived) and a `refreshToken` (long-lived).

Endpoint: POST /api/login

Content-Type: application/json

Request Body:

```
{
  "mobile_number": "9876543210",
  "password": "securepassword123"
}
```

(Note: You can pass either email, mobile_number, or identifier along with the password)

Success Response (200 OK)

```
{
  "success": true,
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR... ",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR... ",
  "user": {
    "id": 123,
    "email": "farmer@example.com",
    "username": "Farmer_User",
    "user_type": 3
  }
}
```

1.3 Verify Token (Me API)

Used to check if the currently stored access token is still valid and fetch user details.

Endpoint: GET /api/user/me

Headers: Authorization: Bearer <accessToken>

Success Response (200 OK)

```
{
  "success": true,
  "message": "Token is valid",
  "user": {
    "id": 123,
    "mobile_number": "9876543210",
    "user_type": 3,
    "profile": { ... }
  }
}
```

1.4 Refresh Token API

Used to get a new pair of tokens when the `accessToken` expires.

Endpoint: POST /api/refresh-token

Headers: Authorization: Bearer <refreshToken> (Note: This endpoint specifically requires the refresh token!)

Success Response (200 OK)

```

{
  "success": true,
  "message": "Access token and refresh token renewed successfully",
  "token": "NEW_ACCESS_TOKEN",
  "refreshToken": "NEW_REFRESH_TOKEN"
}

```

2. Android App Authentication Architecture

To provide a seamless user experience, the Android application should handle authentication centrally. Here is the recommended implementation flow for the Android Developer:

2.1 Storage

- Store both the `accessToken` and `refreshToken` securely in `EncryptedSharedPreferences` or `DataStore`.

2.2 Splash Screen Flow (App Launch)

When the user opens the app, the Splash Screen should determine which screen to show:

1. Check if `accessToken` exists in local storage.
2. If **No**: Navigate immediately to LoginScreen.
3. If **Yes**: Call the `GET /api/user/me` API.
 - If **200 OK**: The token is valid. Navigate to HomeScreen.
 - If **401/403 Unauthorized**: The token has expired. The network interceptor (explained below) will automatically attempt to refresh it. If the refresh fails, clear local storage and navigate to LoginScreen.

2.3 Network Interceptor (OkHttp / Retrofit)

Instead of attaching the token manually to every API call, the Android developer should create an `Interceptor` and an `Authenticator` in OkHttp.

A. Auto-Attaching the Token (Interceptor)

Create an `AuthInterceptor` that intercepts every outgoing request and adds the token header:

```

class AuthInterceptor(private val tokenManager: TokenManager) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val requestBuilder = chain.request().newBuilder()
        tokenManager.getAccessToken()?.let { token ->
            requestBuilder.addHeader("Authorization", "Bearer $token")
        }
        return chain.proceed(requestBuilder.build())
    }
}

```

B. Auto-Refreshing the Token (Authenticator)

When an `accessToken` expires, the backend will return a **401 Unauthorized** error. OkHttp's `Authenticator` catches this globally, pauses the original request, calls the Refresh API, and then retries the original request seamlessly.

```

class TokenAuthenticator(
    private val tokenManager: TokenManager,
    private val apiService: Provider<AuthApiService> // Use Provider to avoid circular dependency
) : Authenticator {

    override fun authenticate(route: Route?, response: Response): Request? {
        // Prevent infinite loops if refresh token itself is rejected
        if (response.request.header("Authorization")?.contains(tokenManager.getRefreshToken() ?: "") == true) {
            tokenManager.clearTokens()
            // Broadcast intent to force user to Login Screen
            return null
        }

        synchronized(this) {
            val currentToken = tokenManager.getAccessToken()

            // Call the refresh API synchronously
            val refreshResponse = apiService.get().refreshToken(
                "Bearer ${tokenManager.getRefreshToken()}"
            ).execute()

            if (refreshResponse.isSuccessful) {
                val newTokens = refreshResponse.body()
                tokenManager.saveTokens(newTokens.token, newTokens.refreshToken)

                // Retry the failed request with the new token
                return response.request.newBuilder()
                    .header("Authorization", "Bearer ${newTokens.token}")
                    .build()
            } else {
                // Refresh token expired or invalid, force logout
                tokenManager.clearTokens()
                return null
            }
        }
    }
}

```

Summary of the Flow:

1. Developer calls `api.getFarmerBookings()`. No token is passed manually.
2. `AuthInterceptor` catches it and attaches: `Authorization: Bearer <accessToken>`.
3. If backend returns **200 OK**, data is returned to the UI.
4. If backend returns **401 Unauthorized** (`accessToken` expired):
 - `TokenAuthenticator` pauses the request.
 - It fires a synchronous request to `POST /api/refresh-token` using the `refreshToken`.
 - Backend returns new tokens.
 - `TokenAuthenticator` saves the new tokens locally.
 - `TokenAuthenticator` retries the original `getFarmerBookings()` request with the *new* access token.
 - The UI never knows an error occurred; it just receives the data slightly delayed.